

Configuración de Servicios: Almacenamiento en Red (NFS) y Autofs

1. Teoría: Almacenamiento Centralizado en Red

NFS (Network File System) es un protocolo de la capa de aplicación diseñado para compartir directorios a través de una red local o de almacenamiento. Su función principal es permitir que un cliente monte una carpeta remota alojada en un servidor y la maneje a nivel de sistema de archivos como si estuviera conectada localmente a su propio disco.

En la administración de infraestructura actual, NFS no se utiliza para compartir archivos de uso personal, sino para resolver problemas de centralización de datos, algunos usos son:

Clústeres Web (Load Balancing): Cuando varios servidores web atienden detrás de un balanceador de carga, todos deben leer y escribir en la misma carpeta de contenido (por ejemplo, imágenes cargadas por usuarios) para mantener la consistencia sin duplicar archivos.

Entornos de Virtualización (Proxmox / VMware): Los discos virtuales de las máquinas virtuales (VMs) se almacenan en un almacenamiento NFS centralizado. Si un hipervisor físico falla, otro nodo del clúster puede levantar la VM de inmediato porque el disco no depende del hardware caído.

Contenedores (Docker / Kubernetes): Permite adjuntar volúmenes persistentes a contenedores efímeros que pueden destruirse y volver a levantarse en diferentes nodos de la red sin perder información.

2. Evolución: NFSv3 vs. NFSv4

Generalmente al hablar de NFS, se listan varios daemons auxiliares para el funcionamiento del servicio:

1. **nfsd** : Proceso principal del servidor NFS.
2. **mountd** : Gestiona las solicitudes iniciales de montaje.
3. **lockd** : Controla el bloqueo de archivos para evitar que dos clientes sobrescriban el mismo bloque al mismo tiempo.
4. **statd** : Monitorea el estado para recuperación tras reinicios.
5. **rpcbind** (o **portmapper**): Mapea los servicios RPC con los puertos del sistema.

Qué cambió en producción/actualidad?

Esta arquitectura multiproceso pertenece a NFSv3. El problema operativo de NFSv3 es que depende de rpcbind (puerto UDP/TCP 111) y asigna puertos aleatorios y dinámicos para el resto de los servicios. En una red con firewalls activos, esto obligaba a abrir rangos enteros de puertos, generando un agujero de seguridad bastante grande.

En entornos actuales se fuerza el uso de NFSv4. Esta versión elimina la necesidad de rpcbind y de los demonios de bloqueo externos, concentrando todo el tráfico (autenticación, montaje, transferencia y estado) en un único puerto estándar, el TCP 2049. Esto simplifica el filtrado mediante tablas de firewall y mejora la estabilidad de la conexión.

3. Vectores de Riesgo y Autenticación por UID

NFS tiene la particularidad de que en su configuración estándar (sec=sys), no valida nombres de usuario ni contraseñas. En su lugar, confía en dos parámetros que viajan en la cabecera del paquete:

1. La dirección IP del cliente.
2. El UID (User ID) y GID (Group ID) del proceso que solicita el acceso.

El problema de la desincronización de UIDs

Linux asigna los permisos de archivos según números identificadores (UIDs), no por el nombre del usuario.

Si en la máquina cliente el usuario juan tiene el UID 1001 y crea un archivo en el directorio NFS, el servidor guardará ese archivo asignándole como dueño el UID 1001. Si en el servidor el UID 1001 pertenece a maria, para el servidor el archivo será de maria.

Para evitar este comportamiento en producción, los servidores y clientes que comparten NFS deben estar integrados a un directorio activo centralizado (como LDAP o FreeIPA) o contar con una asignación estricta de UIDs.

4. Buenas Prácticas de Producción y Hardening

Para desplegar un servidor NFS, se deben tomar las siguientes acciones de seguridad en la configuración de exportaciones (/etc/exports):

A. Mapeo de privilegios (Squashing)

1. `root_squash` (Opción por defecto y obligatoria): Si el usuario root de una máquina cliente intenta escribir en el recurso compartido, el servidor NFS invalida sus privilegios administrativos y lo mapea al usuario anónimo sin permisos nobody (o `nfsnobody`). Evita que un cliente comprometido tome el control del almacenamiento.
2. `no_root_squash`: Desactiva la protección anterior. Permitir acceso root remoto es una falla grave de seguridad y solo se justifica en escenarios muy puntuales como arranque por red (PXE).
3. `all_squash`: Mapea a todos los usuarios clientes (también normales) al usuario anónimo. Muy útil para carpetas de depósito público.

B. Integridad de los datos

1. `sync`: Obliga al servidor a confirmar la escritura en el disco físico antes de responder al cliente que la operación se completó. Pero pierde un poco de velocidad frente a `async`, previene la pérdida o corrupción de datos ante un corte repentino de energía en el storage.
2. `no_subtree_check`: Desactiva la verificación de subárboles de directorios. Mejora el rendimiento del servidor y evita problemas cuando se renombran archivos abiertos.

C. Cifrado y Autenticación Fuerte

En redes no aisladas o donde el tráfico pueda ser interceptado, la validación por IP y UID resulta insuficiente frente a ataques de *IP Spoofing*. La solución de hardening avanzada consiste en integrar NFSv4 con Kerberos (`sec=krb5p`), lo que exige un ticket criptográfico para establecer la sesión y cifra todo el tráfico de datos en tránsito. Además jamás se debe exportar un recurso usando el comodín `*`. La lista de acceso debe limitar explícitamente la dirección IP individual o la subred exacta permitida.

D. Gestión de Puntos de Montaje en Clientes (Autofs)

Definir montajes de NFS en el archivo estático `/etc/fstab` del cliente es una mala práctica operativa. Si el servidor de almacenamiento se reinicia o sufre una caída de red, el sistema cliente se bloqueará intentando acceder a una ruta inexistente (*Stale file handle*), dejando congelados procesos e instrucciones de la terminal.

La alternativa técnica correcta es implementar Autofs:

1. Un demonio en el cliente monitorea los directorios de montaje.
 2. El recurso de red se conecta automáticamente en el instante exacto en que un usuario o proceso intenta ingresar a la carpeta.
 3. Tras un período configurable de inactividad (por ejemplo, 60 segundos), Autofs desmonta la unidad de forma limpia, liberando sockets de red y evitando cuelgues en el cliente.
-

Práctica de laboratorio NFSv4

En este laboratorio usaremos 2 maquinas, una es el host (pc fisica) y la otra es el servidor, en mi caso, usare el servidor donde ya hemos configurado DNS, SSH, SFTP (estos servicios se encuentran documentados en mi repositorio de github)

Paso 1: preparacion del Servidor de Storage

1. **Primero, nos conectamos al servidor que va a alojar los archivos e instalamos el motor del servicio.**

```
~ apt update  
~ apt install nfs-kernel-server
```

En Linux, NFS puede correr en espacio de usuario o directamente integrado en el núcleo (kernel). Instalamos nfs-kernel-server porque al operar a nivel de kernel, los accesos a disco físico y las transferencias de red son mucho más rápidas y consumen menos CPU.

2. **Creamos el directorio que vamos a compartir y le asignamos los permisos:**

```
~ mkdir -p /srv/nfs/storage_compartido  
~ chown nobody:nogroup /srv/nfs/storage_compartido  
~ chmod 775 /srv/nfs/storage_compartido
```

```
root@UTN-Lucka:~# mkdir -p /srv/nfs/storage_compartido  
root@UTN-Lucka:~# chown nobody:nogroup /srv/nfs/storage_compartido  
root@UTN-Lucka:~# chmod 775 /srv/nfs/storage_compartido
```

Esto es **obligatorio para la seguridad**, por que como vimos en la teoría, vamos a aplicar el `root_squash`. Si el usuario `root` del equipo cliente intenta escribir algo aca, el servidor lo obliga a hacerlo bajo el usuario `nobody`. Si la carpeta no le pertenece a `nobody`, directamente no podrá escribir.

Paso 2: Configuración de la Lista de Control de Acceso (/etc/exports)

1. En El archivo /etc/exports definimos quién entra y cómo.

~ vim /etc/exports

Agregamos la siguiente línea (cambiando por tu IP y mascara):

/srv/nfs/storage_compartido

192.168.100.0/24(rw,sync,no_subtree_check,root_squash)

```
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/srv/nfs/storage_compartido 192.168.122.0/24(rw,sync,no_subtree_check,root_squash)
```

Desglose de la configuracion

/srv/nfs/storage_compartido: Es la ruta física en nuestro servidor que estamos publicando.

IP/Mascara: Control de acceso por IP. Solo las máquinas de esta subred pueden comunicarse. **Es importante** nunca usar * (asterisco), ya que dejaría el disco abierto a cualquier red enrutada hacia el servidor.

rw: permite operaciones de lectura y escritura.

sync: hace que el servidor al vaciar la caché de memoria y escribir físicamente en el disco antes de avisarle al cliente que la transferencia terminó. Protege los datos contra cortes eléctricos.

no_subtree_check: Le indica al motor NFS que no pierda tiempo verificando si un archivo modificado sigue perteneciendo al árbol de directorios original. Gana mucho rendimiento y evita desconexiones si alguien renombra una carpeta del lado del servidor.

root_squash: saca los privilegios de cualquier usuario remoto que intente operar como root.

Paso 3: aplicación en caliente y firewall

1. Para que el servidor empiece a publicar la carpeta, ponemos la configuración en la memoria de NFS.

Nota: en vez de usar `systemctl` que pararía el servicio de manera brusca y desconectaría a los usuarios que estuvieran usándolo, es mejor usar **exportfs**, este comando sincroniza el archivo. `-a` (todo), `-r` (re-exporta), `-v` (muestra en pantalla lo que hizo).

~ **exportfs -arv**

si ese comando no funciona, como es mi caso, podemos usar:

~ **/usr/sbin/exportfs -arv**

(variara dependiendo de la manera que escalamos privilegios a root)

```
root@UTN-Lucka:~# /usr/sbin/exportfs -arv
exporting 192.168.122.20/24:/srv/nfs/storage_compartido
```

2. Habilitamos el servicio en el arranque y cerramos el puerto en el firewall:

~ `systemctl enable --now nfs-kernel-server`

~ `ufw allow from IP/MASC to any port 2049 proto tcp`

```
luck4@UTN-Lucka:~$ su -
Password:
root@UTN-Lucka:~# ufw allow ssh
Rules updated
Rules updated (v6)
root@UTN-Lucka:~# ufw allow from 192.168.122.20/24 to any port 2049 proto tcp
WARN: Rule changed after normalization
Rules updated
root@UTN-Lucka:~# ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

Hardening: Exigimos el uso del puerto TCP 2049. Esto fuerza (implícitamente) a los clientes a usar NFSv4, bloqueando el tráfico desorganizado y los puertos aleatorios de NFSv3.

Paso 4: Configuración del Cliente y Montaje Dinámico (Autofs)

1. Nos vamos ahora a la máquina del cliente (192.168.100.20). Aca no instalamos el servidor, solo las utilidades de cliente y el daemon Autofs.

~ **apt update**

~ **apt install nfs-common autofs**

```
root@Luck4-Debian:~# apt install nfs-common autofs
nfs-common is already the newest version (1:2.8.3-1).
autofs is already the newest version (5.1.9-1.2+b1).
```

2. Vamos a decirle a Autofs en qué directorio raíz va a trabajar editando su archivo maestro.

~ **vim /etc/auto.master**

Agregamos al final del archivo:

/mnt/nfs /etc/auto.nfs --timeout=60 --ghost

Desglose de la configuracion:

/mnt/nfs: directorio local donde apareceran los discos de red.

/etc/auto.nfs: archivo secundario donde detallaremos las rutas exactas.

--timeout=60: Si ningún proceso del cliente lee o escribe en el disco durante 60 segundos, Autofs cierra los sockets TCP y desmonta la unidad automáticamente.

--ghost: Crea directorios vacíos falsos. Si no lo ponemos, el usuario no verá la carpeta hasta que escriba explícitamente su nombre, haciendolo poco intuitivo.

3. Ahora creamos el archivo secundario con los parámetros de la conexión NFS.

~ **vim /etc/auto.nfs**

Definimos la conexión hacia el servidor:

storage -fstype=nfs4,rw,soft,intr 192.168.100.10:/srv/nfs/storage_compartido

Desglose:

storage: El nombre de la carpeta final. La ruta completa será /mnt/nfs/storage.

fstype=nfs4: Forzamos el uso de la versión segura del protocolo.

soft,intr: NFS usa montajes tipo hard. Si el servidor remoto se apaga o hay un corte, un montaje hard bloqueará infinitamente la terminal del cliente esperando a que el servidor vuelva. Las opciones soft e intr (interruptible) le dicen al kernel que, tras cierto tiempo sin respuesta, aborte la operación y devuelva un simple "Error de Entrada/Salida" sin colgar el sistema operativo cliente.

4. Iniciamos el servicio de montaje bajo demanda:

~ **systemctl restart autofs**

~ **systemctl enable autofs**

```
root@Luck4-Debian:~# systemctl restart autofs
root@Luck4-Debian:~# systemctl enable autofs
Synchronizing state of autofs.service with SysV service script
Executing: /usr/lib/systemd/systemd-sysv-install enable autofs
root@Luck4-Debian:~#
```

Paso 5: Auditoría y Comprobación

1. Para probar la eficiencia de esta arquitectura, primero listamos los discos duros activos.

~ **df -h | grep nfs**

No devuelve ninguna salida. Porque el disco NO está conectado.

Ingresamos al directorio. Ahora, Autofs captura la petición, negocia con el servidor y hacer el montaje de forma invisible para el usuario.

~ **cd /mnt/nfs/storage**

~ **df -h | grep nfs**

```
root@Luck4-Debian:~# cd /mnt/nfs/storage
root@Luck4-Debian:/mnt/nfs/storage# df -h | grep nfs
192.168.122.20:/srv/nfs/storage_compartido 19G 1.7G 16G 10% /mnt/n
root@Luck4-Debian:/mnt/nfs/storage#
```

Si esperamos 60 segundos fuera del directorio, el recurso se desmontará solo, comprobando que nuestra configuración de gestión de recursos funciona a la perfección.

Nota: En un entorno NFSv4 con Firewall, herramientas clásicas como showmount -e van a fallar (Timeout) porque intentan usar el puerto 111. Para probar la conexión real, siempre hacer un montaje manual antes de configurar Autofs: mount -v -t nfs4 IP:/ruta /ruta_local.

Conclusión:

En este laboratorio vemos que montar almacenamiento en red no es solo compartir una carpeta. Descartamos la inseguridad de NFSv3, forzamos tráfico por un único puerto (2049) para asegurar el Firewall, mitigamos la escalada de privilegios con `root_squash` y optimizamos los recursos de red del cliente usando montaje bajo demanda con `autofs`.